Документація програми Pulso PPG Backend

Детальний опис роботи системи збору фотоплетизмографічного сигналу, обробки IR/Red каналів, розрахунку BPM і SpO2, API, сховища даних, математичних формул, фізичних принципів і використаних технологій.

Python + FastAPI | ESP32 + MAX30102 | PPG signal processing | BPM / SpO2 estimation | PostgreSQL optional persistence

Проєкт	`pulso_py_new`
Робочий шлях	`/Users/vitkarino/Desktop/pulso_py_new`
Стан коду	Поточний локальний стан репозиторію, commit `6dbf7ee`
Дата документа	27 квітня 2026 року
Мова	Українська

Важливо. Описана програма є інженерним прототипом для збору та аналізу PPG-сигналу. Результати BPM і SpO2 залежать від калібрування, посадки датчика, рухових артефактів, перфузії тканин, оптичної геометрії та електронних налаштувань. Документ не стверджує клінічну придатність системи без окремої валідації.

Зміст

1. Призначення та межі системи
2. Архітектура програми
3. Використані технології
4. Потік даних від датчика до результату
5. HTTP та WebSocket API
6. Збереження записів і модель даних
7. Фізична основа PPG та пульсоксиметрії
8. Алгоритми цифрової обробки сигналу
9. Математична модель BPM, SpO2 і якості сигналу
10. Логіка прошивки ESP32
11. Конфігурація та базові значення
12. Перевірка, тестування та обмеження
13. Перелік посилань

1. Призначення та межі системи

Pulso PPG Backend приймає від ESP32 пакети фотоплетизмографічних зразків з двох оптичних каналів: інфрачервоного `ir` та червоного `r`. Сервер підтримує живу обробку потоку, короткі вимірювальні сесії, довші записи, трансляцію сирих пакетів для інтерфейсу, збереження записів у PostgreSQL та експорт даних у JSON або CSV.

Основні вихідні фізіологічні оцінки: частота серцевих скорочень `bpm`, насичення крові киснем `spo2`, ratio-of-ratios `ratio`, індекс довіри `sensor_confidence` і структурований опис якості сигналу `signal_quality`. Значення можуть бути `null`, якщо вікно ще коротке, контакт з пальцем не виявлений або сигнал не дає надійної AC/DC оцінки.

Функціональна межа: система оцінює життєві показники з PPG-сигналу на рівні програмного прототипу. Вона не містить клінічного калібрування під конкретну оптичну конструкцію, пацієнтську популяцію, температуру, тон шкіри, рух або медичний стандарт перевірки точності.

2. Архітектура програми

Проєкт складається з двох частин: Python-бекенду та прошивки ESP32. Бекенд приймає WebSocket-повідомлення, валідує JSON, веде ковзні буфери, фільтрує PPG-канали, рахує метрики та керує записами. Прошивка налаштовує MAX30102/MAX3010x, підключається до Wi-Fi, відкриває WebSocket-клієнт і передає батчі зразків.

Логічна схема обробки

MAX30102
Red/IR LED, фотодетектор, FIFO, температура.

ESP32
I2C-зчитування, прорідження до 25 Гц, JSON batch.

WebSocket
`/ws` або `/ws/esp32`, hello/start/stop.

Processing
Буфер, Hampel, detrend, Butterworth, peaks, Welch.

API/DB
Метрики, записи, live chart, CSV/JSON export.

Файл або модуль	Роль у системі
`app/main.py`	Створює FastAPI-додаток, REST endpoints, WebSocket endpoints, CSV/JSON експорт.
`app/processing/service.py`	Оркеструє валідацію JSON, буферизацію, фільтрацію, розрахунок метрик і записи.
`app/processing/buffer.py`	Зберігає ковзне вікно сигналу окремо для кожного пристрою.
`app/processing/filters.py`	Виконує Hampel suppression, лінійний detrend і Butterworth band-pass.
`app/processing/metrics.py`	Рахує BPM, SpO2, perfusion index, confidence і рівень якості сигналу.
`app/measurement.py`	Керує сесіями вимірювання та записами з фіксованою або відкритою тривалістю.
`app/recording_repository.py`	Описує таблиці SQLAlchemy Core і операції з PostgreSQL/сумісною БД.
`app/websocket_handler.py`	Підтримує WebSocket-з'єднання пристроїв, live subscribers і команди start/stop.
`embedded/pulso_esp32/pulso_esp32.ino`	Прошивка ESP32 для MAX30102, Wi-Fi, WebSocket та пакетної передачі PPG.

3. Використані технології

Backend

Технологія	Як використовується
Python	Основна мова серверної логіки, математичних розрахунків і тестів.
FastAPI	HTTP API, WebSocket маршрути, автоматична OpenAPI-сумісна структура endpoint-ів.
Uvicorn	ASGI-сервер для запуску FastAPI-додатку, HTTP/1.1 і WebSocket обробки.
Pydantic v2	Валідація вхідних JSON-пакетів, моделей пристрою, зразків і вихідних метрик.
NumPy	Масиви сигналу, середні, медіани, RMS, стандартне відхилення, IQR.
SciPy Signal	Фільтр Butterworth, detrend, zero-phase filtering, peak finding, Welch PSD.
SQLAlchemy Core	Опис таблиць, insert/update/select/delete, індекси, транзакції.
psycopg / PostgreSQL	Опційне постійне збереження користувачів, проєктів, записів і sample rows.

Embedded

Технологія	Як використовується
Arduino/C++ для ESP32	Цикл `setup()` / `loop()`, робота з Wi-Fi, I2C і таймерами `millis()`.
MAX30105 / MAX3010x library	Ініціалізація MAX30102, FIFO, Red/IR samples, temperature read.
ArduinoJson	Серіалізація батчів PPG у компактний JSON і розбір команд backend-а.
arduinoWebSockets	WebSocket client для з'єднання ESP32 із FastAPI backend-ом.

4. Потік даних від датчика до результату

4.1. Формат вхідного пакета

Сервер очікує JSON, де кореневий об'єкт має поле `device`. Кожен sample містить інфрачервоний канал `ir` і червоний канал `r`. Частота дискретизації передається в полі `fs`.

```
{
  "device": {
    "id": "F4:65:0B:55:2E:80",
    "recording_id": "optional-recording-uuid",
    "temp": 31.75,
    "fs": 25,
    "samples": [
      {"ir": 84231, "r": 53211,
    ]
  }
}
```

4.2. Валідація

Pydantic-моделі `DevicePayload`, `DeviceData` і `PPGSample` перевіряють, що ідентифікатор пристрою непорожній, `fs > 0`, фактична частота не нижча за 5 Гц, а зразки каналів додатні. Поле `r` в JSON мапиться на внутрішнє поле `red`.

4.3. Ковзне вікно

Для кожного `device_id` створюється окремий `DeviceSignalBuffer`. Якщо пристрій змінює `fs`, буфер очищається, бо зразки з різними частотами не можна коректно змішувати в одному часовому ряді. Максимальна довжина вікна: `round(MAX_WINDOW_SECONDS * fs)`. За стандартних значень це `20 * 25 = 500` samples.

4.4. Результат метрик

Успішно оброблений пакет повертає `VitalSigns` і оновлює `MetricsStore`. Під час активного запису backend також додає sample rows у сесію та може транслювати live batch через `/recordings/live`.

```
{
  "device_id": "F4:65:0B:55:2E:80",
  "timestamp": "2026-04-27T07:42:29Z",
  "fs": 25,
  "temperature": 31.75,
  "bpm": 71.4,
  "spo2": 96.0,
  "ratio": 0.5547,
  "sensor_confidence": 0.969,
  "signal_quality":
    "level": "high",
    "samples_in_window": 375,
    "window_seconds": 15.0,
    "perfusion_index": 3.5426,
    "peak_count": 18,
    "reason": null
  }
}
```

5. HTTP та WebSocket API

5.1. Сервісні endpoints

Endpoint	Призначення
`GET /health`	Перевірка доступності сервера та поточного порту.
`GET /metrics`	Останні метрики по всіх пристроях у пам'яті процесу.
`GET /metrics/{device_id}`	Останні метрики конкретного пристрою або 404.
`GET /devices`	Підключені WebSocket-пристрої та кількість live subscribers.

5.2. Вимірювання та записи

Endpoint	Логіка
`POST /measurements/{device_id}/start`	Стартує коротку сесію для пристрою. За замовчуванням тривалість 15 с.
`GET /measurements`	Повертає всі активні або завершені сесії в пам'яті процесу.
`GET /measurements/{device_id}`	Стан сесії конкретного пристрою.
`POST /api/recordings/start`	Створює запис, очищає буфер пристрою, надсилає ESP32 команду `start`.
`POST /api/recordings/{id}/stop`	Зупиняє запис у backend-і та надсилає пристрою команду `stop`.
`GET /api/recordings`	Список записів із фільтрами `limit`, `offset`, `date_from`, `date_to`.
`GET /api/recordings/{id}/samples`	Посторінкове читання sample rows конкретного запису.
`GET /api/recordings/{id}/extract?format=json|csv`	Експорт одного запису з samples.
`GET /api/recordings/extract?format=json|csv`	Експорт усіх записів із фільтрацією за датою.

5.3. WebSocket

Endpoint	Призначення
`ws://host:8080/ws`	Основний канал прийому PPG JSON від пристрою або клієнта.
`ws://host:8080/ws/esp32`	Еквівалентний канал для ESP32-прошивки.
`ws://host:8080/recordings/live`	Підписка UI на live-пакети зразків активного запису.

ESP32 спершу надсилає `{"type":"hello","device_id":"..."}`. Backend реєструє WebSocket під цим device id і відповідає `hello_ack`. Коли HTTP-клієнт стартує запис, backend надсилає на цей WebSocket команду `start` з `recording_id`. Після локального завершення або команди stop пристрій повертає `finished`, `start_ack` або `stop_ack`.

6. Збереження записів і модель даних

PostgreSQL вмикається, якщо заданий `DATABASE_URL`, `POSTGRES_DSN` або компонентні змінні `POSTGRES_HOST`, `POSTGRES_PORT`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`. Без БД live-метрики й сесії працюють у пам'яті, але endpoints для recordings/projects/users повертають 503.

Таблиця	Ключові поля	Призначення
`users`	`id`, `name`, `age`, `sex`, `recordings_qty`	Люди, для яких можуть створюватися записи.
`projects`	`id`, `title`, `description`, `recordings_qty`	Групування записів за дослідженням або проєктом.
`project_users`	`project_id`, `user_id`, `assigned_at`	Зв'язок many-to-many між користувачами та проєктами.
`recordings`	`id`, `started_at`, `finished_at`, `bpm`, `spo2`, `signal_quality`	Метадані запису та фінальний результат обробки.
`recordings_samples`	`recording_id`, `sample_index`, `raw_data`, `created_at`	Сирі IR/Red samples кожного запису.

У `recordings_samples` є унікальне обмеження `(recording_id, sample_index)`, що не дає випадково записати однаковий sample index двічі. Індекси за `started_at`, `status` і `recording_id` прискорюють типові запити.

7. Фізична основа PPG та пульсоксиметрії

7.1. Фотоплетизмографія

Фотоплетизмографія вимірює зміну інтенсивності світла, що проходить через тканину або відбивається від неї. Коли серце викидає кров, артеріальний об'єм у периферичній тканині періодично змінюється; це змінює поглинання та розсіювання світла. PPG-сигнал умовно складається з повільної непульсуючої складової DC і малої пульсуючої складової AC, синхронної з серцевим циклом [1].

У цій системі MAX30102 працює як reflective PPG sensor: червоний та інфрачервоний світлодіоди освітлюють тканину, а фотодетектор вимірює відбите світло. Сенсор також має FIFO, програмовані параметри sample rate, LED current, pulse width, ADC range та внутрішній температурний сенсор [5].

7.2. Чому потрібні два канали

Оксигемоглобін і дезоксигемоглобін мають різні спектри поглинання в червоній та інфрачервоній області. Пульсоксиметрія використовує це співвідношення: зміни AC/DC на двох довжинах хвиль комбінуються у ratio-of-ratios, після чого SpO2 отримують через емпіричну калібрувальну функцію [2], [3].

Фізичний сенс AC/DC:
DC приблизно відповідає сталому оптичному шляху через тканини, венозну кров, непульсуючу артеріальну кров і конструктивні умови датчика.
AC відповідає зміні оптичного сигналу, викликаній серцевим пульсом.
$PI = (AC_{IR} / DC_{IR}) * 100\%$.

7.3. Перфузійний індекс

Perfusion index є відношенням пульсуючої частини сигналу до непульсуючої частини. Він не є самостійним медичним діагнозом, але добре підходить як інженерний індикатор сили PPG-сигналу та якості контакту. У програмі PI використовується для contact detection, дозволу/заборони SpO2 та внеску в `sensor_confidence` [4].

8. Алгоритми цифрової обробки сигналу

8.1. Hampel suppression для одиничних викидів

Перед фільтрацією кожен канал проходить Hampel-подібну обробку. Для кожного sample у центрі локального вікна розраховується медіана та robust scale через median absolute deviation.

Якщо точка відхиляється більше ніж на `3 * MAD`, її замінюють медіаною. Це приглушує одиничні spike-и без сильного спотворення форми пульсу [7].

Для вікна W_i:
$m_i = \text{median}(W_i)$
$MAD_i = 1.4826 * \text{median}(|x_j - m_i|)$
якщо $|x_i - m_i| > 3 * MAD_i$, тоді $x_i := m_i$.

8.2. Лінійний detrend

PPG має повільний baseline drift через рух, тиск пальця, дихання, температуру та зміну контакту. `scipy.signal.detrend(..., type="linear")` віднімає найкращу лінійну складову з поточного вікна, залишаючи швидші коливання, корисні для BPM і AC-оцінки.

$x_d[n] = x_{clean}[n] - (a\,n + b)$, де a і b оцінюються як лінійний тренд на поточному вікні.

8.3. Butterworth band-pass

Після detrend сигнал проходить смуговий фільтр Butterworth 4-го порядку зі стандартною смугою `0.5..3.0 Hz`. Верхня частота обмежується `0.95 * Nyquist`, щоб не порушити цифрові умови фільтрації. Звичайний режим використовує `sosfiltfilt`, тобто прямий і зворотний прохід для нульової фазової затримки; якщо вікно замале, код переходить на `sosfilt` [6], [9].

$f_{Nyquist} = f_s / 2$.
Для $f_s = 25$ Hz маємо $f_{Nyquist} = 12.5$ Hz, тому смуга 0.5..3.0 Hz безпечна для цифрової обробки.
3.0 Hz відповідає 180 bpm, бо bpm = 60 * f.

8.4. Пошук піків

Піки шукаються на відфільтрованому IR-каналі. Мінімальна відстань між піками задається через максимальний дозволений пульс: `round(fs * 60 / MAX_BPM)`. При `fs=25` і `MAX_BPM=180` це приблизно 8 samples. Початкова prominence дорівнює `0.35 * std(filtered_ir)`. Якщо піків замало, код робить повторний пошук із relaxed threshold на основі MAD.

8.5. Welch fallback

Якщо часовий пошук піків не дає достатньо валідних інтервалів, система переходить до спектральної оцінки через метод Welch. У діапазоні `MIN_BPM/60 .. MAX_BPM/60` вибирається частота з найбільшою потужністю, але тільки якщо пік щонайменше втричі перевищує median noise floor. Додаткова перевірка не дає алгоритму зафіксуватися на другій гармоніці, якщо біля `f/2` є порівнянний пік [8], [9].

9. Математична модель BPM, SpO2 і якості сигналу

9.1. Мінімальне вікно

Якщо довжина вікна менша за `MIN_WINDOW_SECONDS`, система не видає BPM і SpO2. За стандартних значень потрібно мінімум 8 секунд. Confidence у цьому стані масштабується лише до 0.2, а `signal_quality.level` дорівнює `warming_up`.

$$\text{window_seconds} = N / f_s$$

якщо window_seconds < MIN_WINDOW_SECONDS, тоді confidence = clamp(window_seconds / MIN_WINDOW_SECONDS * 0.2).

9.2. BPM з міжпікових інтервалів

Якщо знайдено достатньо піків, оцінка BPM базується на медіані валідних inter-beat intervals. Інтервали фільтруються за межами пульсу `MIN_BPM..MAX_BPM`, а потім очищаються від outlier-ів через IQR-правило.

$$IBI_k = (peak_{k+1} - peak_k) / f_s$$

valid IBI: $60 / MAX_BPM <= IBI_k <= 60 / MIN_BPM$

$$BPM = 60 / \text{median}(\text{valid IBI})$$

Регулярність ритму оцінюється через coefficient of variation:

$$CV = \text{std}(IBI) / \text{mean}(IBI)$$

$$\text{regularity_score} = \text{clamp}(1 - CV / 0.18)$$

9.3. AC/DC для SpO2

DC-компоненти беруться як середні значення сирих каналів. AC-компоненти оцінюються за медіанною амплітудою між піком і мінімумом наступного сегмента. Якщо піків недостатньо, використовується RMS fallback: `2 * sqrt(2) * RMS(filtered)`. Для синусоїдального сигналу це наближує peak-to-peak amplitude; у ratio-of-ratios однаковий множник у двох каналах скорочується.

$$DC_{IR} = \text{mean}(\text{raw_ir}),\ DC_{Red} = \text{mean}(\text{raw_red})$$

$$R = (AC_{Red} / DC_{Red}) / (AC_{IR} / DC_{IR})$$

$$PI = (AC_{IR} / DC_{IR}) * 100$$

9.4. Калібрувальна формула SpO2

У поточному коді SpO2 рахується квадратичною емпіричною формулою з коефіцієнтами з конфігурації. Це типова інженерна форма для двохвильової pulse oximetry, але її потрібно калібрувати під конкретний датчик, LED current, геометрію корпуса, місце вимірювання та референсний пульсоксиметр [2], [3].

```
SpO2 = clamp(0, 100, A R^2 + B R + C + offset)
Стандартно: A = -45.060, B = 30.354, C = 94.845, offset = -1.8.
```

9.5. Контакт і якість сигналу

Contact detection захищає від вимірювань без пальця, із засвіченим або насиченим сенсором. Якщо DC нижчий за мінімум, канал насичений або PI за межами очікуваного діапазону, система повертає `no_contact`, `bpm=null`, `spo2=null` і `sensor_confidence=0`.

Порогове значення	Стандарт	Сенс
`MIN_IR_DC`	50000	IR DC занизький: немає достатнього оптичного контакту.
`MIN_RED_DC`	10000	Red DC занизький.
`MAX_SENSOR_DC`	250000	Канал імовірно насичений.
`MIN_PERFUSION_INDEX`	0.05%	Мінімум для виявлення контакту.
`MIN_SPO2_PERFUSION_INDEX`	0.05%	Мінімум для публікації SpO2.
`MAX_PERFUSION_INDEX`	20%	Надто великий PI: нестабільний або засвічений сигнал.

9.6. Sensor confidence

Confidence є евристичним score від 0 до 1. Він не є статистичною ймовірністю. Формула комбінує тривалість вікна, perfusion index, регулярність інтервалів, кількість піків і правдоподібність ratio.

```
confidence = 0.20 * window_score + 0.25 * pi_score + 0.30 * regularity_score + 0.15 *
peak_score + 0.10 * ratio_score
якщо BPM відсутній: confidence := confidence * 0.45
якщо SpO2 відсутній: confidence := confidence * 0.70
```

Рівень `signal_quality.level` стає `high`, якщо є BPM і SpO2, confidence не нижче 0.7, PI достатній і знайдено щонайменше 6 піків. Інакше система повертає `medium` або `low`.

10. Логіка прошивки ESP32

10.1. Налаштування сенсора

Прошивка використовує MAX30105-compatible API для сенсора MAX30102. Поточні параметри: LED brightness `0x1F`, sample average `4`, режим двох LED `2`, hardware sample rate `100 Hz`, reported sample rate `25 Hz`, pulse width `411 us`, ADC range `4096`.

Сенсор генерує FIFO-зразки швидше, ніж backend отримує дані. ESP32 вичитує FIFO, але додає у batch лише ті samples, які відповідають програмному інтервалу `1000 / 25 = 40 ms`. Це дає backend-у стабільну заявлену частоту 25 Гц.

10.2. WebSocket lifecycle

1. `setup()` ініціалізує Serial, MAX30102, Wi-Fi та WebSocket.
2. Після `WStype_CONNECTED` пристрій надсилає `hello`.
3. Backend реєструє device id і може пізніше надіслати `start`.
4. Команда `start` задає `activeRecordingId`, тривалість і очищає FIFO.
5. Під час streaming ESP32 накопичує 25 samples і відправляє batch JSON.
6. Коли локальний wall-clock duration завершився, ESP32 відправляє останній batch і `finished`.
7. Команда `stop` примусово відправляє поточний batch, повертає `stop_ack` і чистить стан.

10.3. Пакетування

`batchSize=25` означає приблизно один JSON-пакет на секунду при 25 Гц. Це зменшує overhead WebSocket-повідомлень і водночас залишає backend-у достатньо часте оновлення live-графіків.

```
{
  "device": {
    "id": "F4:65:0B:55:2E:80",
    "recording_id": "uuid",
    "fs": 25,
    "temp": 25.6,
    "samples": [
      {"ir": 133000, "r": 99300,
    ]
  }
}
```

10.4. Відновлення після збоїв

Якщо Wi-Fi втрачено, прошивка зупиняє streaming без очищення `activeRecordingId` і пробує перепідключитися кожні 5 секунд. WebSocket має heartbeat `15 s / 3 s / 2 retries`. Якщо

WebSocket-відправка не вдалася, streaming також зупиняється, щоб не накопичувати неконсистентний стан.

11. Конфігурація та базові значення

Конфігурація backend-а читається з environment variables, а файл `.env` підвантажується простим локальним парсером до створення `AppConfig`. Якщо змінної немає, використовується стандартне значення з `app/config.py`.

Параметр	Стандарт	Вплив
`MIN_BPM` / `MAX_BPM`	40 / 180	Фізіологічні межі BPM для peak intervals і Welch band.
`BANDPASS_LOW_HZ` / `BANDPASS_HIGH_HZ`	0.5 / 3.0	Смуга фільтра PPG для виділення серцевої компоненти.
`FILTER_ORDER`	4	Порядок Butterworth-фільтра.
`MIN_WINDOW_SECONDS`	8.0	Мінімальна довжина сигналу перед видачею повних метрик.
`MAX_WINDOW_SECONDS`	20.0	Максимальна довжина ковзного live-вікна.
`MEASUREMENT_DURATION_SECONDS`	15.0	Тривалість короткої сесії за замовчуванням.
`MIN_PEAKS`	4	Мінімальна кількість піків для часової BPM-оцінки.
`SPO2_A/B/C/OFFSET`	-45.060 / 30.354 / 94.845 / -1.8	Емпірична квадратична калібровка SpO2.

Запуск сервера:

```
python3 -m uvicorn app.main:app --host 0.0.0.0 --port 8080
```

12. Перевірка, тестування та обмеження

12.1. Автоматичні тести

У `tests/test_processing.py` створюється синтетичний PPG-сигнал: синусоїда з заданим BPM, DC-рівнями, AC-амплітудами, ratio і шумом. Тести перевіряють основний сценарій, warming-up, no-contact, низький PI для SpO2, завершення measurement session, matching recording id і live payload.

Під час підготовки документа виконано:

```
python3 -m unittest tests/test_processing.py
```

Результат: **9 tests, OK**. Для синтетичного 15-секундного сигналу тестовий прогін отримав приблизно `71.4 BPM`, `SpO2 96.0%`, `ratio 0.5547`, `sensor_confidence 0.969`.

12.2. Інженерні обмеження

- SpO2 є оцінкою, а не клінічно валідованим значенням; коефіцієнти треба калібрувати на референсних даних.
- Алгоритм не має акселерометра або adaptive motion artifact cancellation, тому рух може створювати хибні піки.
- Двохвильова модель не відрізняє карбоксигемоглобін, метгемоглобін та інші нестандартні оптичні стани крові.
- Вплив тону шкіри, температури, вазоконстрикції та низької перфузії не компенсується окремою моделлю.
- WebSocket/REST endpoints у поточному коді не мають автентифікації й авторизації.
- При перезапуску процесу in-memory `MetricsStore` і активні сесії втрачаються; записи зберігаються тільки якщо налаштована БД.
- CSV-експорт розгортає recording-level fields на кожен sample row; це зручно для аналізу, але дублює метадані.

12.3. Рекомендації для подальшої валідації

- Зібрати парні вимірювання з сертифікованим пульсоксиметром і підігнати `SPO2_A/B/C/OFFSET`.
- Додати motion quality score або акселерометричний канал для відкидання рухових артефактів.
- Логувати firmware version, LED current, contact pressure proxy і умови вимірювання разом із записом.
- Додати інтеграційні тести для WebSocket команд start/stop і PostgreSQL persistence.
- Ввести автентифікацію для API, якщо система використовується поза локальною лабораторною мережею.

13. Перелік посилань

1. Allen J. *Photoplethysmography and its application in clinical physiological measurement*. Physiological Measurement, 28(3), R1-R39, 2007. https://doi.org/10.1088/0967-3334/28/3/R01
2. Jubran A. *Pulse oximetry*. Critical Care, 19, Article 272, 2015. https://doi.org/10.1186/s13054-015-0984-8
3. Mendelson Y. *Pulse Oximetry: Theory and Applications for Noninvasive Monitoring*. Clinical Chemistry, 38(9), 1601-1607, 1992. https://doi.org/10.1093/clinchem/38.9.1601
4. Coutrot M., Dudoignon E., Joachim J., Gayat E., Vallee F., Depret F. *Perfusion index: Physical principles, physiological meanings and clinical implications in anaesthesia and critical care*. Anaesthesia Critical Care & Pain Medicine, 40(6), 100964, 2021. https://doi.org/10.1016/j.accpm.2021.100964
5. Analog Devices / Maxim Integrated. *MAX30102 High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health*. https://www.analog.com/en/products/max30102.html
6. Butterworth S. *On the Theory of Filter Amplifiers*. Experimental Wireless and the Wireless Engineer, 7, 536-541, 1930. https://cir.nii.ac.jp/crid/1572261550684507392
7. Hampel F. R. *The Influence Curve and its Role in Robust Estimation*. Journal of the American Statistical Association, 69(346), 383-393, 1974. https://doi.org/10.1080/01621459.1974.10482962
8. Welch P. D. *The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms*. IEEE Transactions on Audio and Electroacoustics, 15(2), 70-73, 1967. https://doi.org/10.1109/TAU.1967.1161901
9. SciPy Documentation. *Signal processing (scipy.signal)*. https://docs.scipy.org/doc/scipy/reference/signal.html
10. NumPy Documentation. *ndarray*. https://numpy.org/devdocs/reference/generated/numpy.ndarray.html
11. FastAPI Documentation. https://fastapi.tiangolo.com/
12. Uvicorn Documentation. https://www.uvicorn.org/
13. Pydantic Documentation. *Models*. https://docs.pydantic.dev/latest/concepts/models/
14. SQLAlchemy 2.0 Documentation. https://docs.sqlalchemy.org/en/20/
15. SparkFun. *SparkFun MAX3010x Sensor Library*. https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library
16. ArduinoJson Documentation. *serializeJson() and deserializeJson()*. https://arduinojson.org/v6/api/json/serializejson/, https://arduinojson.org/v6/api/json/deserializejson/

17. Markus Sattler. *arduinoWebSockets*. https://github.com/Links2004/arduinoWebSockets